VTU COMPUTER ORGANIZATION & ARCHITECTURE

MODULE 4

Memory System

Topic	Key Contents
1. Basic Concepts	MAR, MDR, Address space, Memory Access Time, Memory Cycle Time
2. Semiconductor RAM	Memory chip organization, word-line, sense/write circuit
3. Static RAM (SRAM)	Latch circuit, T1/T2 transistors, CMOS cell, Read/Write operations
4. Asynchronous DRAM	Capacitor storage, RAS/CAS signals, refresh, fast page mode
5. Synchronous DRAM (SDRAM)	Clock sync, row/column latches, burst read, timing diagram
6. Latency, Bandwidth & DDR	Performance metrics, DDR-SDRAM, SIMM/DIMM
7. Memory Controller & Rambus	Address multiplexing, RAS/CAS generation, RDRAM, packets
8. Read Only Memory (ROM)	ROM cell, PROM, EPROM, EEPROM, Flash Memory, Flash Cards/Drives
9. Speed, Size & Cost	Memory hierarchy: Registers → L1 → L2 → Main → Disk
10. Cache Memory	Locality of reference, mapping functions, write protocols
11. Direct Mapping	Block-j mod 128, tag/set/word fields, cache hit/miss
12. Associative Mapping	Any block to any location, 12-bit tag, flexible placement
13. Set-Associative Mapping	Sets + tag, valid bit, dirty bit, k-way set associative
14. Replacement Algorithm	LRU algorithm, block counter, hit/miss handling
15. Performance & Interleaving	Hit rate, miss penalty, avg access time, interleaved modules
16. Virtual Memory	MMU, virtual/physical address, page table, page frame
17. Address Translation & TLB	Page table base register, TLB, page faults, LRU
18. Memory Management	System/user space, privileged instructions, page protection
19. Secondary Storage – Magnetic	Tracks, sectors, cylinders, seek time, latency, Winchester

Disk	
20. Disk Organization & Controller	Disk access, ECC, disk controller functions, SCSI/DMA
21. Solved Problems (9)	DRAM refresh, SDRAM transfer, cache bits, virtual memory
22. Quick Revision Summary	All key facts, formulas, and tables

TABLE OF CONTENTS

1. Basic Concepts of Memory System

Maximum memory size that can be used in a computer is determined by the addressing mode:

MODULE 4: MEMORY SYSTEM

BASIC CONCEPTS

• Maximum size of memory that can be used in any computer is determined by addressing mode.

Address	Memory Locations
16 Bit	2^{16} = 64 K
32 Bit	2^{32} = 4G (Giga)
40 Bit	2^{40} = 1T (Tera)

MAR (Memory Address Register): If MAR is k-bits long, memory may contain up to 2^k addressable locations.

MDR (Memory Data Register): If MDR is n-bits long, then n bits of data are transferred between memory and processor in one operation.

Processor Bus: Data transfer takes place over the processor bus (Figure 8.1), which has 3 sets of lines:

- **Address Lines:** Carry the address of the memory location to be accessed.
- **Data Lines:** Carry the actual data being read or written.
- **Control Lines:** Carry control signals such as R/W' (read/write) and MFC (Memory Function Completed) for coordinating data transfer.

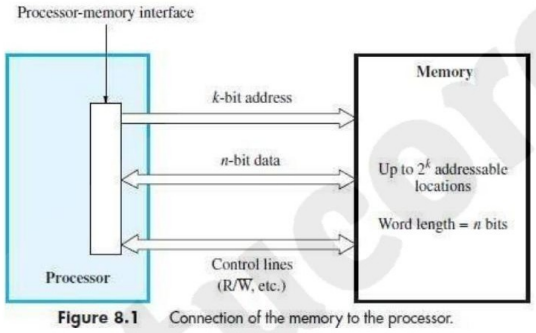

Figure 8.1 Connection of the memory to the processor.

- If MAR is k-bits long then
  → memory may contain upto 2^K addressable-locations
- If MDR is n-bits long, then
  → n-bits of data are transferred between the memory and processor.
- The data-transfer takes place over the processor-bus (Figure 8.1).
- The processor-bus has
  1) Address-Line
  2) Data-line &
  3) Control-Line (R/W", MFC – Memory Function Completed).
- The Control-Line is used for coordinating data-transfer.
- The processor reads the data from the memory by
  → loading the address of the required memory-location into MAR and
  → setting the R/W" line to 1.
- The memory responds by

Figure 8.1 — Connection of memory to the processor (Processor-Memory Interface)

Read Operation:

1. **Processor loads address** of required memory location into MAR.
2. **Processor sets R/W' = 1** (indicating a read operation).
3. **Memory places data** from the addressed location onto the data lines.
4. **Memory asserts MFC signal** (Memory Function Completed) to confirm data is ready.
5. **Processor loads data** from the data lines into MDR.

Write Operation:

6. **Processor loads address** of target location into MAR.
7. **Processor sets R/W' = 0** (indicating a write operation).
8. **Processor places data** to be written onto the data lines.
9. **Memory writes data** from data lines into the addressed location.

Memory Access Time: The time that elapses between the INITIATION of a memory operation and the COMPLETION of that operation. It measures how long it takes to read or write one word.

Memory Cycle Time: The MINIMUM time delay required between the initiation of two SUCCESSIVE memory operations. Cycle time >= Access time (additional time needed to restore circuitry).

> *VTU Key: Memory Access Time = time for one operation. Memory Cycle Time = minimum gap between two consecutive operations. Cycle time is always >= Access time.*

2. Semiconductor RAM Memories – Internal Organization

Memory Cell Array: Memory cells are organized in the form of an ARRAY (Figure 8.2). Each cell stores ONE BIT of information.

Key structural elements:

- **Each row of cells** forms a memory WORD (all bits of one word are in the same row).
- **Word-Line:** All cells of a row are connected to a common line called the Word-Line. Activating this line selects the entire row (word).
- **Sense/Write Circuits:** The cells in each column are connected to a Sense/Write circuit by 2 bit-lines (b and b'). These circuits read data during reads and write data during writes.
- **Bidirectional Data Line:** Data input and output of each Sense/Write circuit are connected to a single bidirectional data line that connects to the computer's data bus.

Control Signals:

- **R/W' (Read/Write):** Specifies the operation — 1 for read, 0 for write.
- **CS' (Chip Select):** Selects a given chip in a multi-chip memory system. Only the selected chip responds to the processor.

Write operation: The Sense/Write circuit receives input information and stores it in the cells of the selected word. The word-line is activated, the bit-lines are driven to the required state, and the cell latches the value.

• During a write-operation, the sense/write circuit
→ receive input information &
→ store input info in the cells of the selected word.

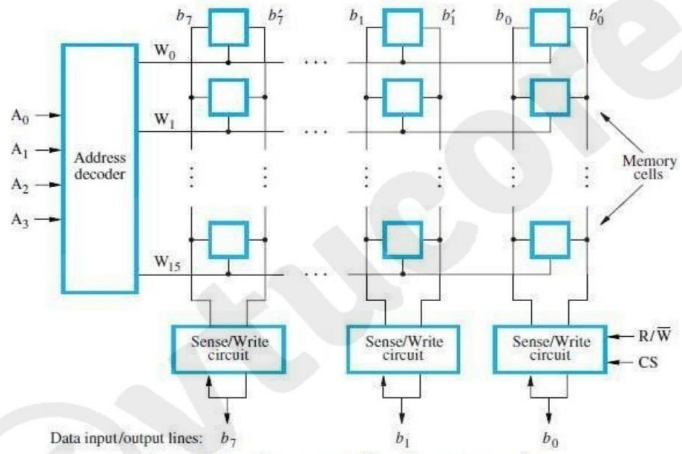

Figure 8.2 Organization of bit cells in a memory chip.

• The data-input and data-output of each Sense/Write circuit are connected to a single bidirectional data-line.
• Data-line can be connected to a data-bus of the computer.
• Following 2 control lines are also used:
1) R/W' → Specifies the required operation.
2) CS' → Chip Select input selects a given chip in the multi-chip memory-system.

Bit Organization	Requirement of external connection for address, data and control lines
128 (16x8)	14
(1024) 128x8(1k)	19

Figure 8.2 — Organization of bit cells in a memory chip (16-word x 8-bit example)

3. Static RAM (SRAM)

Definition: Memories consisting of circuits capable of retaining their state as long as power is applied are called Static RAM (SRAM). They use a LATCH (two cross-coupled inverters) as the storage element.

Circuit structure:

- Two inverters are cross-connected to form a LATCH (bistable circuit that holds 0 or 1)
- The latch is connected to two bit-lines (b and b') by transistors T1 and T2
- T1 and T2 act as switches controlled by the WORD-LINE
- When the word-line is at ground level → T1 and T2 are OFF → latch retains its state indefinitely (as long as power is on)

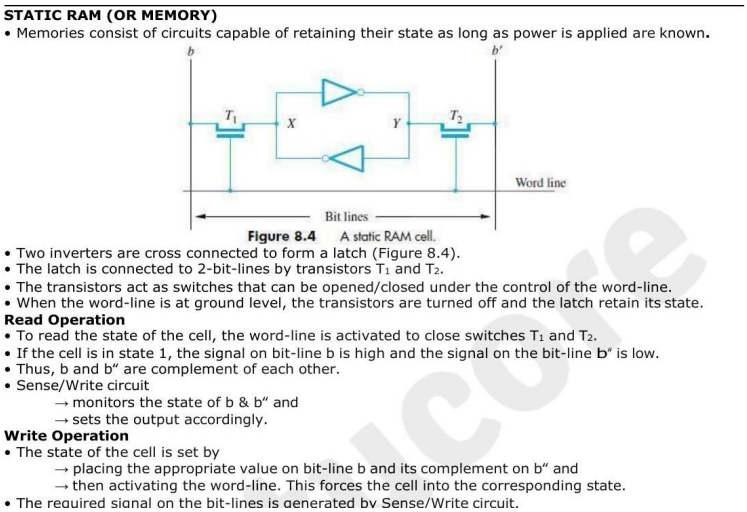

Figure 8.4 — A static RAM cell (cross-coupled inverters forming a latch, T1/T2 access transistors)

3.1 Read Operation

10. **Activate word-line** → This turns ON transistors T1 and T2 (closing the switches).
11. **If cell is in state 1:** bit-line b goes HIGH, bit-line b' goes LOW (they are complements).
12. **If cell is in state 0:** bit-line b goes LOW, bit-line b' goes HIGH.
13. **Sense/Write circuit monitors b and b'** and sets the output data accordingly.

3.2 Write Operation

14. **Sense/Write circuit drives bit-lines:** Places appropriate value on b and its complement on b'.
15. **Activate word-line** → forces the cell latch into the required state.
16. **After word-line deactivated** → latch retains the new state.

3.3 CMOS Memory Cell

CMOS implementation: Transistor pairs (T3, T5) and (T4, T6) form the two inverters in the latch (Figure 8.5).

- **State 1:** Voltage at X is HIGH → T5, T6 are ON → T3, T4 are OFF. T1, T2 are ON → b is HIGH, b' is LOW.

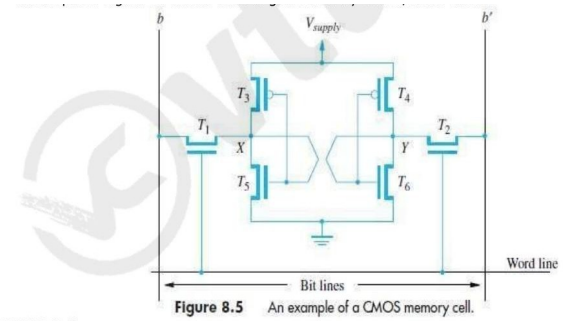

Figure 8.5 An example of a CMOS memory cell.

CMOS Cell

- Transistor pairs (T_3, T_5) and (T_4, T_6) form the inverters in the latch (Figure 8.5).
- In state 1, the voltage at point X is high by having T_5, T_6 ON and T_4, T_5 are OFF.
- Thus, T_1 and T_2 returned ON (Closed), bit-line b and b" will have high and low signals respectively.
- **Advantages:**
  1) It has low power consumption „." the current flows in the cell only when the cell is active.
  2) Static RAM"s can be accessed quickly. It access time is few nanoseconds.
- **Disadvantage:** SRAMs are said to be volatile memories „." their contents are lost when power is interrupted.

3-4

Figure 8.5 — An example of a CMOS memory cell (6 transistors)

Advantages of SRAM:

- **Low power consumption:** Current flows only when the cell is being accessed (CMOS draws minimal static current).
- **Fast access time:** Access time is a few nanoseconds — very fast.

Disadvantage: SRAM is VOLATILE — contents are LOST when power is interrupted. Also, SRAM chips are large (6 transistors per cell) and expensive compared to DRAM.

4. Asynchronous DRAM

Motivation: Less expensive RAMs can be implemented using simpler cells with only ONE transistor and ONE capacitor per bit (instead of 6 transistors for SRAM).

Storage mechanism: Information is stored as a CHARGE on a capacitor (Figure 8.6). The presence of charge = logic 1, absence = logic 0.

Key characteristic — Dynamic:

- **The charge on the capacitor leaks away** over time (tens of milliseconds). Without intervention, data would be lost.
- **Contents must be periodically REFRESHED** by restoring the capacitor charge to its full value. This is done by reading and rewriting each row periodically.

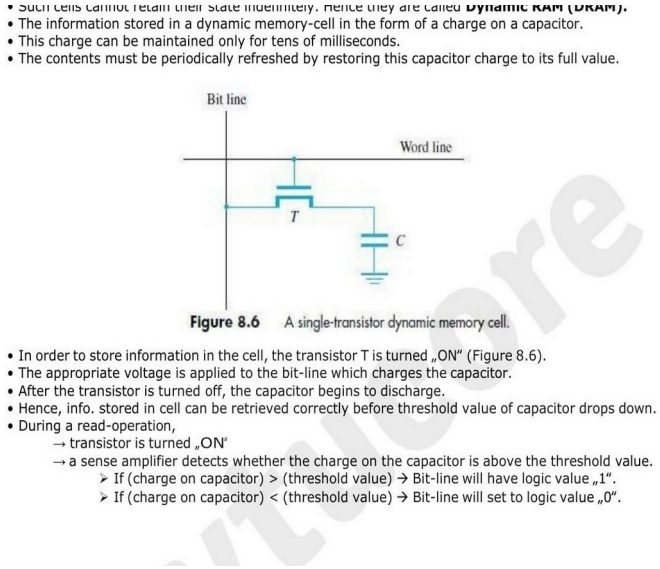

Figure 8.6 — A single-transistor dynamic memory cell (transistor T + capacitor C)

Write operation:

17. **Turn ON transistor T** (by activating the word-line).
18. **Apply appropriate voltage to bit-line** → charges the capacitor.
19. **Turn OFF transistor T** → capacitor begins to slowly discharge (leakage).

Read operation:

20. **Turn ON transistor T.**
21. **Sense amplifier detects capacitor charge:**
    - **Charge > threshold** → bit-line logic 1.
    - **Charge < threshold** → bit-line logic 0.
22. **Reading is destructive** → must restore (refresh) the cell after each read.

4.1 DRAM Internal Organization & Operation

Example: 2M x 8 DRAM chip (Figure 5.7). 4096 bit cells per row, divided into 512 groups of 8.

21-bit address needed (to access one byte), split into:

- **12 bits (A8-A0):** Row address → selects one of 4096 rows.
- **9 bits (A20-A9):** Column address → selects one of 512 groups of 8 bits in the selected row.

Address is sent in TWO phases (multiplexed address):

23. **Row Address (RAS'):** Row address is applied first and loaded into the Row Address Latch in response to the RAS' (Row Address Strobe) signal going LOW. When a Read is initiated, ALL cells in the selected row are read and refreshed.
24. **Column Address (CAS'):** Shortly after, column address is applied to the same address pins and latched by CAS' (Column Address Strobe) going LOW.

25. **Data transfer:** R/W'=1 → read: output values transferred to data lines D0-D7. R/W'=0 → write: data from D0-D7 written to selected cells.

Important: RAS' and CAS' are ACTIVE-LOW signals — they cause address latching when they transition from HIGH to LOW.

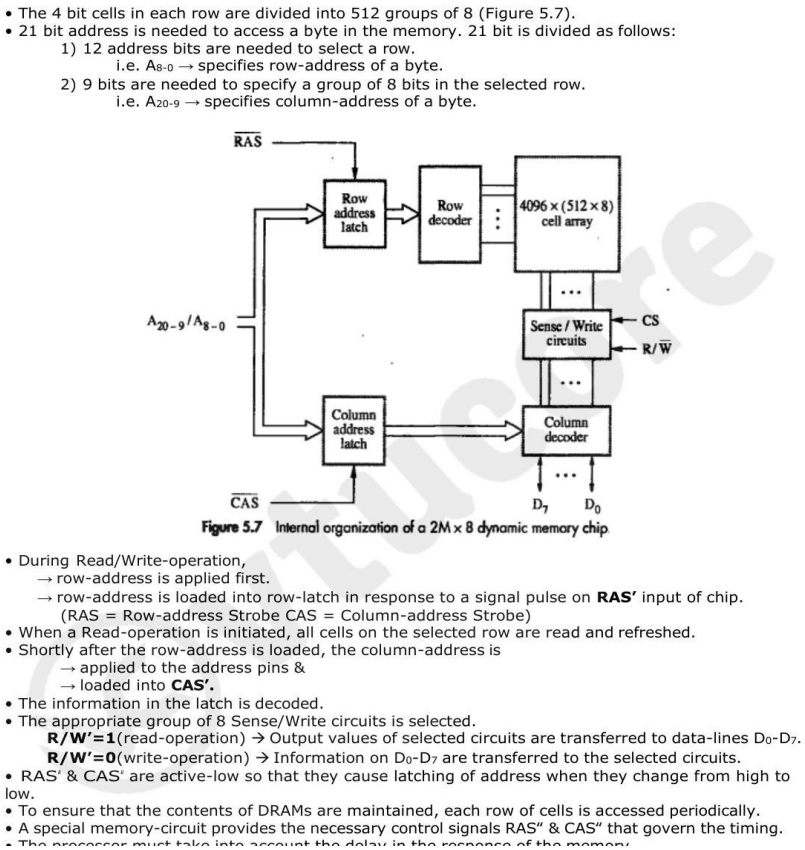

Figure 5.7 — Internal organization of a 2M x 8 dynamic memory chip (RAS/CAS, Row/Column address latch, Cell array, Sense/Write)

VTU Key: DRAM uses multiplexed address — row address first (latched by RAS), then column address (latched by CAS). Only half as many address pins needed. This is why DRAM chips can have more storage capacity for fewer pins.

4.2 Fast Page Mode

Fast Page Mode is a performance enhancement for DRAM:

- After selecting a row (via RAS'), multiple bytes can be read from the SAME row by sending consecutive column addresses under control of successive CAS' signals — WITHOUT reissuing RAS'
- This allows transferring a block of data at a much faster rate since the slow row selection step is done only once
- Called "page" mode because one row of the DRAM is called a page

Advantage: Burst transfers from one row are very fast — only CAS latency for subsequent words in the same row.

5. Synchronous DRAM (SDRAM)

Key difference from Asynchronous DRAM: All operations are directly SYNCHRONIZED with the system CLOCK signal (Figure 8.8).

Enhanced features of SDRAM:

- **Registered inputs:** Address and data connections are buffered by registers — all inputs are sampled at clock edges.
- **Sense amplifier latches:** Output of each sense amplifier is connected to a latch. A Read causes contents of ALL cells in selected row to be loaded into these latches simultaneously.
- **Data output register:** Data held in latches for selected columns is transferred to the data-output register, making data available on output pins at clock edges.
- **Column-address counter:** SDRAM automatically increments the column address to access the next set of bits — supporting burst reads.

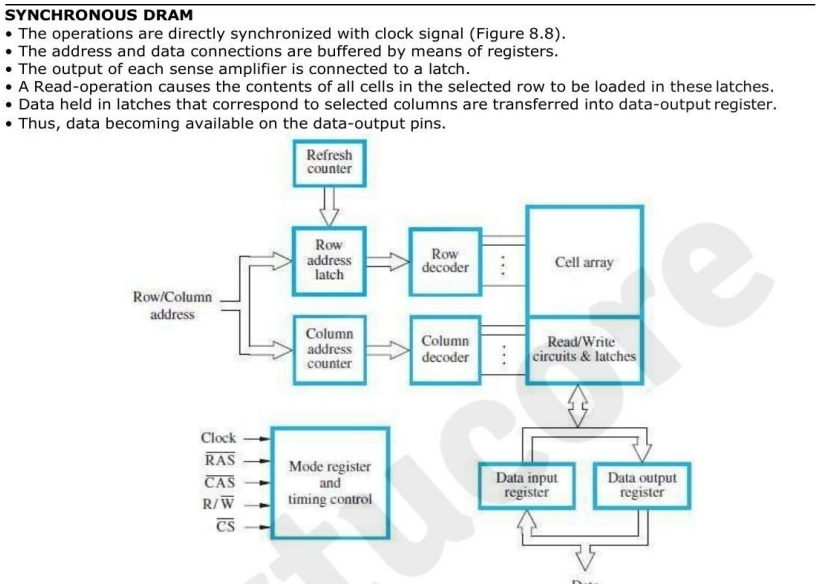

Figure 8.8 — Synchronous DRAM (SDRAM) internal architecture

SDRAM Read Sequence (Figure 8.9 — Burst of 4 words):

26. **Row address latched** under control of RAS' signal.
27. **Memory takes 2-3 clock cycles** to activate (sense) the selected row.
28. **Column address latched** under control of CAS' signal.
29. **After CAS latency (1 clock cycle),** first data word (D0) appears on data lines.
30. **SDRAM auto-increments column address** → D1, D2, D3 appear on successive clock edges.

Figure 8.8 Synchronous DRAM.

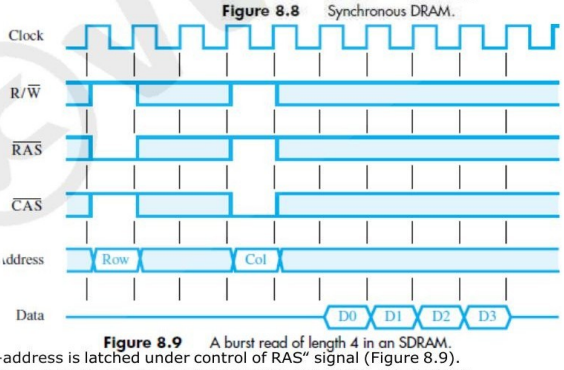

Figure 8.9 A burst read of length 4 in an SDRAM.

- First, the row-address is latched under control of RAS" signal (Figure 8.9).
- The memory typically takes 2 or 3 clock cycles to activate the selected row.
- Then, the column-address is latched under the control of CAS" signal.
- After a delay of one clock cycle, the first set of data bits is placed on the data-lines.
- SDRAM automatically increments column-address to access next 3 sets of bits in the selected row.

3-7

Figure 8.9 — Burst read of length 4 in SDRAM (Clock, R/W, RAS, CAS, Address Row/Col, Data D0-D3)

VTU: SDRAM burst read = 1 RAS + 1 CAS + auto-increment for remaining words. Much faster than individual reads. Clock synchronization allows pipelining and predictable timing.

6. Latency, Bandwidth & DDR-SDRAM

Two key performance metrics:

- **Latency:** The amount of TIME it takes to transfer a SINGLE WORD of data to or from memory. For single-word transfers, latency is the complete performance indicator. For block transfers, it denotes the time to transfer the FIRST WORD of data.
- **Bandwidth:** The NUMBER OF BITS OR BYTES that can be transferred per second. Bandwidth = (word size in bits) / (time per word). Bandwidth depends on: (1) speed of access to stored data AND (2) number of bits accessed in parallel.

6.1 Double Data Rate SDRAM (DDR-SDRAM)

Standard SDRAM: Transfers data only on the RISING edge of the clock signal.

DDR-SDRAM innovation: Transfers data on BOTH the rising AND falling edges of the clock (both edges used → double the data rate at same clock frequency).

Key DDR features:

- **Doubled bandwidth** for long burst transfers — same clock speed, twice the data rate.
- **Cell array organized into two banks:** Each bank accessed separately. Consecutive words stored in different banks.
- **Interleaving:** Two words accessed simultaneously from different banks and transferred on successive clock edges.

6.2 Structure of Larger Memories – SIMM & DIMM

Physical implementation: Large memory systems are built from DRAM chips arranged in MEMORY MODULES.

Why modules? Placing many individual DRAM chips directly on the motherboard wastes space and makes design complex. Modules provide a compact, standardized form factor.

Module Type	Full Name	Description
SIMM	Single Inline Memory Module	Memory chips on a small PCB that plugs into a socket on motherboard. 32-bit wide data path.
DIMM	Dual Inline Memory Module	Like SIMM but with independent contacts on both sides. 64-bit wide data path. Current standard.
RIMM	Rambus Inline Memory Module	Uses Rambus channel technology. High-bandwidth narrow bus.

7. Memory Controller & Rambus Memory

Memory Controller

Why needed: Dynamic memory chips use MULTIPLEXED ADDRESS INPUTS to reduce the number of pins. The full address is split into two halves and sent sequentially.

Address multiplexing:

- **High-Order Address Bits (Row Address):** Sent FIRST, latched into memory chips under control of RAS' signal.
- **Low-Order Address Bits (Column Address):** Sent SECOND on the SAME address pins, latched using CAS' signal.

Memory Controller functions (Figure 5.11):

- Accepts complete address and R/W' signal from the processor
- Forwards row portion and column portion of address sequentially to memory
- Generates RAS' and CAS' timing signals
- Sends R/W' and CS' signals to memory
- A Request signal indicates that a memory access is needed

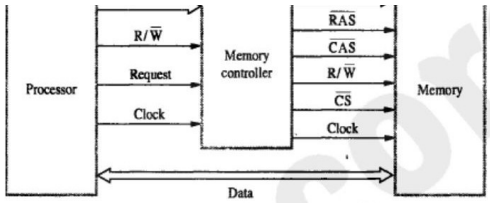

Figure 5.11 Use of a memory controller.

- The Controller accepts a complete address & R/W" signal from the processor.
- A Request signal indicates a memory access operation is needed.
- Then, the Controller
  - → forwards the row & column portions of the address to the memory.
  - → generates RAS" & CAS" signals &
  - → sends R/W" & CS" signals to the memory.

RAMBUS MEMORY

- The usage of wide bus is expensive.
- Rambus developed the implementation of narrow bus.
- Rambus technology is a fast signaling method used to transfer information between chips.
- The signals consist of much smaller voltage swings around a reference voltage V_{ref}.
- The reference voltage is about 2V.
- The two logical values are represented by 0.3V swings above and below V_{ref}.
- This type of signaling is generally is known as **Differential Signalling**.
- Rambus provides a complete specification for design of communication called as **Rambus Channel**.
- Rambus memory has a clock frequency of 400 MHz.
- The data are transmitted on both the edges of clock so that effective data-transfer rate is 800MHZ.

Figure 5.11 — Use of a memory controller (Address multiplexing, RAS/CAS generation)

Rambus Memory (RDRAM)

Problem with wide buses: Using a wide data bus (64-bit, 128-bit) is expensive — requires many carefully impedance-controlled signal lines on the PCB.

Rambus solution: Implements a NARROW BUS with high-speed differential signaling.

Feature	Details
Signaling	Differential signaling — small voltage swings (0.3V) above and below reference voltage Vref (~2V)
Clock frequency	400 MHz
Effective data rate	800 MHz (transfers on both edges of clock)
Bus width	9 data lines (8 data + 1 parity), no separate address lines
Two-channel	18 data lines total
Packet types	Request, Acknowledge, Data — all communicated via packets on data lines
Chips	RDRAM (Rambus DRAM) — interface circuitry included on chip

8. Read Only Memory (ROM)

Why ROM? Both SRAM and DRAM are VOLATILE — they lose stored information when power is turned off. Many applications require NON-VOLATILE memory that retains data even without power.

Examples of ROM use:

- BIOS firmware in computers (system boot code)
- Operating system loader
- Microcontroller programs in embedded systems (washing machines, automotive ECU)

ROM cell operation (Figure 8.11):

- **Logic 0:** Transistor T is connected to ground at point P → switch closed → voltage on bit-line drops to near zero.
- **Logic 1:** Transistor switch is open → bit-line remains at high voltage.
- To read: activate word-line → sense circuit at end of bit-line generates proper output.

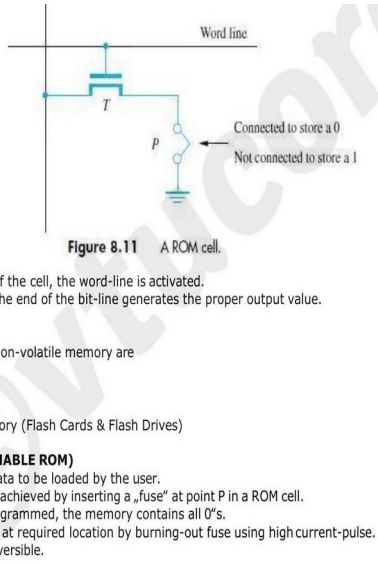

Figure 8.11 — A ROM cell (transistor T with connection point P)

8.1 PROM – Programmable ROM

Key feature: PROM allows data to be loaded (programmed) by the USER — not just at manufacturing time.

How it works:

- Programmability is achieved by inserting a FUSIBLE LINK at point P in each ROM cell
- Before programming: all memory locations contain 0 (all fuses intact)
- To write a 1: user burns out the fuse at that location using a HIGH CURRENT PULSE
- **Irreversible:** Once burned, the fuse cannot be restored — programming is permanent.

Advantages:

- Flexibility — user can program it for their specific application
- Faster than mask ROM programming process at factory
- Less expensive for small quantities — no custom masks needed

8.2 EPROM – Erasable Programmable ROM

Improvement over PROM: EPROM can be ERASED and REPROGRAMMED — not permanently fixed.

Technology:

- Uses a special floating-gate transistor at each cell location

- Transistor can function as a normal transistor OR as a permanently-OFF disabled transistor
- **Programming:** Inject charge into the floating gate by applying a high voltage — transistor becomes permanently OFF (stores 0).
- **Erasure:** Expose the chip to ULTRAVIOLET (UV) LIGHT through a transparent quartz window — UV dissipates the trapped charges, restoring all cells to 1.

Advantages:

- Flexibility during development — reprogram during debugging
- Capable of retaining stored information for a very long time (decades)

Disadvantages:

- Chip must be PHYSICALLY REMOVED from circuit for reprogramming
- ENTIRE contents must be erased by UV light — cannot erase individual bits

8.3 EEPROM – Electrically Erasable ROM

Improvement over EPROM: EEPROM can be both programmed and erased ELECTRICALLY — no UV light, no removing from circuit.

Advantages:

- Can be both programmed and erased electrically while in the circuit (in-system programming)
- Allows SELECTIVE erasing — individual cells or bytes can be erased

Disadvantage: Requires DIFFERENT VOLTAGES for erasing, writing, and reading the stored data — more complex circuitry needed.

8.4 Flash Memory (Flash Cards & Flash Drives)

Flash vs EEPROM:

- **EEPROM:** Can read and write individual cells.
- **Flash:** Can read individual cells, but WRITE (program) an ENTIRE BLOCK at once. Before writing, previous contents of the block must be ERASED.

Flash Applications: USB drives, SD cards, SSDs, MP3 players, smartphones.

Two implementations:

1) Flash Cards

- Flash chips mounted on a small card with a STANDARD INTERFACE
- Simply plugged into a conveniently accessible slot
- Available in 8MB, 32MB, 64MB and larger sizes
- Example: 1 minute of music ≈ 1MB → 64MB card = 1 hour of music

2) Flash Drives (SSD)

- Designed to FULLY EMULATE the hard disk drive — OS sees it as a regular disk
- **SOLID STATE:** No moving parts — purely electronic storage.

Flash Drive Advantages:

- Shorter seek and access time → faster response than mechanical hard disk
- Low power consumption → attractive for battery-powered devices
- Insensitive to vibration and shock

Flash Drive Disadvantages:

- Capacity historically less than hard disk (though modern SSDs have closed this gap)
- Higher cost per bit than magnetic hard disk
- **Write endurance:** Flash memory degrades after approximately 1 million write/erase cycles per block — eventually wears out.

9. Speed, Size & Cost – Memory Hierarchy

Memory Hierarchy: Computers use a hierarchy of memory types, trading off speed, size, and cost:

SPEED, SIZE COST

Characteristics	SRAM	DRAM	Magnetis Disk
Speed	Very Fast	Slower	Much slower than DRAM
Size	Large	Small	Small
Cost	Expensive	Less Expensive	Low price

Memory	Speed	Size	Cost
Registers	Very high	Lower	Very Lower
Primary cache	High	Lower	Low
Secondary cache	Low	Low	Low
Main memory	Lower than Seconadry cache	High	High
Secondary Memory	Very low	Very High	Very High

- The main-memory can be built with DRAM (Figure 8.14)
- Thus, SRAM"s are used in smaller units where speed is of essence

Speed, Size, Cost comparison: SRAM vs DRAM vs Magnetic Disk

- The Cache-memory is of 2 types:
  - **1) Primary/Processor Cache** (Level1 or L1 cache)
    - It is always located on the processor-chip.
  - **2) Secondary Cache** (Level2 or L2 cache)
    - It is placed between the primary-cache and the rest of the memory.
- The memory is implemented using the dynamic components (SIMM, RIMM, DIMM).
- The access time for main-memory is about 10 times longer than the access time for L1 cache.

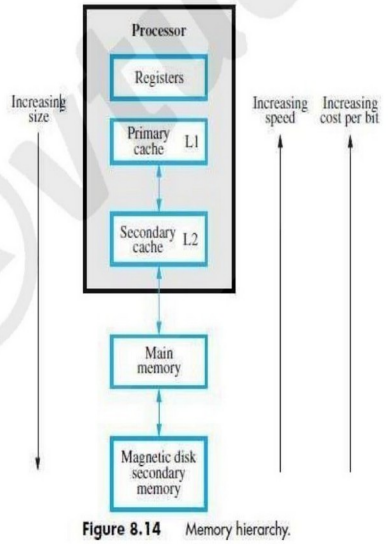

Figure 8.14 Memory hierarchy.

3-12

Figure 8.14 — Memory hierarchy (Processor → Registers → L1 Cache → L2 Cache → Main Memory → Magnetic Disk)

Memory Level	Speed	Size	Cost/bit
Registers (in CPU)	Very High	Very Small	Very High
L1 Cache (Primary)	High	Small	High
L2 Cache (Secondary)	Moderate	Small	Moderate
Main Memory (DRAM)	Lower than L2	Large	Low
Secondary (HDD/SSD)	Very Low	Very Large	Very Low

Key facts about cache types:

- **L1 Cache (Primary/Processor Cache):** Always located ON the processor chip. Fastest, smallest. Access time is typically 1-4 clock cycles.
- **L2 Cache (Secondary Cache):** Located between the primary cache and main memory. Larger and slightly slower than L1. May be on-chip or off-chip.
- **Main Memory access time:** Approximately 10x LONGER than L1 cache access time. This gap makes cache essential for performance.

10. Cache Memories

Purpose: Cache is a small, fast memory inserted BETWEEN the larger, slower main memory AND the processor. It holds the currently active portions of a program and its data.

10.1 Locality of Reference

Cache effectiveness is based on the property called LOCALITY OF REFERENCE:

Two types of locality:

- **Temporal Locality:** Recently executed instructions/data are LIKELY to be accessed AGAIN very soon. (Loop execution reuses the same instructions repeatedly.)
- **Spatial Locality:** Instructions/data in CLOSE PROXIMITY to recently accessed ones are ALSO LIKELY to be accessed soon. (Sequential program execution, arrays accessed in order.)

If the active segment of a program is placed in cache, total execution time can be REDUCED significantly because most accesses are served from the fast cache instead of slow main memory.

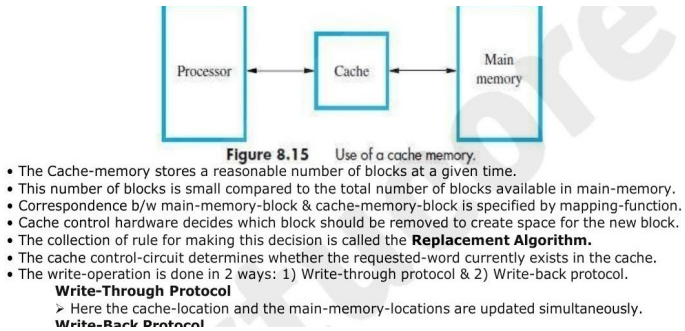

Figure 8.15 — Use of a cache memory (Processor ↔ Cache ↔ Main Memory)

Key terms:

- **Block:** A set of contiguous memory locations of a fixed size (the unit of transfer between main memory and cache).
- **Cache Line:** Synonym for cache block.
- **Mapping Function:** Specifies the correspondence between main-memory blocks and cache-memory positions.
- **Replacement Algorithm:** The rule used by cache control hardware to decide which block to evict when the cache is full and a new block must be loaded.

10.2 Write-Through & Write-Back Protocols

Write-Through Protocol

- **Simultaneous update:** When the processor writes to a cache location, the same data is SIMULTANEOUSLY written to the corresponding main memory location.
- **Advantage:** Main memory is always consistent with cache — no stale data.
- **Disadvantage:** Every write operation requires a slow main memory write — reduces write performance.

Write-Back Protocol

- **Delayed update:** Only the CACHE location is updated immediately. The main memory location is NOT updated.
- **Dirty/Modified bit:** The cache controller marks the updated cache block with a Dirty bit (also called Modified bit) to remember it needs to be written back.
- **Write-back:** When the marked (dirty) block is eventually evicted from cache, it is written back to main memory at that time.

- **Advantage:** Faster — most writes stay in fast cache. Only blocks that were actually modified go back to memory.
- **Disadvantage:** Complexity — cache and memory may be inconsistent. Cache coherence problems with DMA.

Read Miss & Write Miss Handling

Read Miss: When the requested word is NOT in cache (cache miss during read):

- **Load-Through/Early Restart:** The block of words containing the requested word is copied from main memory into cache. The requested word is forwarded to the processor immediately when it arrives (without waiting for the entire block).

Write Miss: When the written address is not in cache:

- **Write-Through:** Write directly to main memory.
- **Write-Back:** First bring the block into cache, then write to the cache copy.

11. Direct Mapping

Rule: Block j of main memory maps to cache block (j mod 128). This means main-memory blocks 0, 128, 256, 384... all map to cache block 0; blocks 1, 129, 257... map to cache block 1; and so on.

Memory address structure (16-bit address, 128-block cache, 16-word blocks):

- **Low-order 4 bits (Word field):** Selects one of 16 words within the block.
- **Next 7 bits (Block field):** Determines which cache position (0-127) the block maps to.
- **High-order 5 bits (Tag field):** The tag stored in cache along with the block — used to identify which main memory block is currently in this cache location.

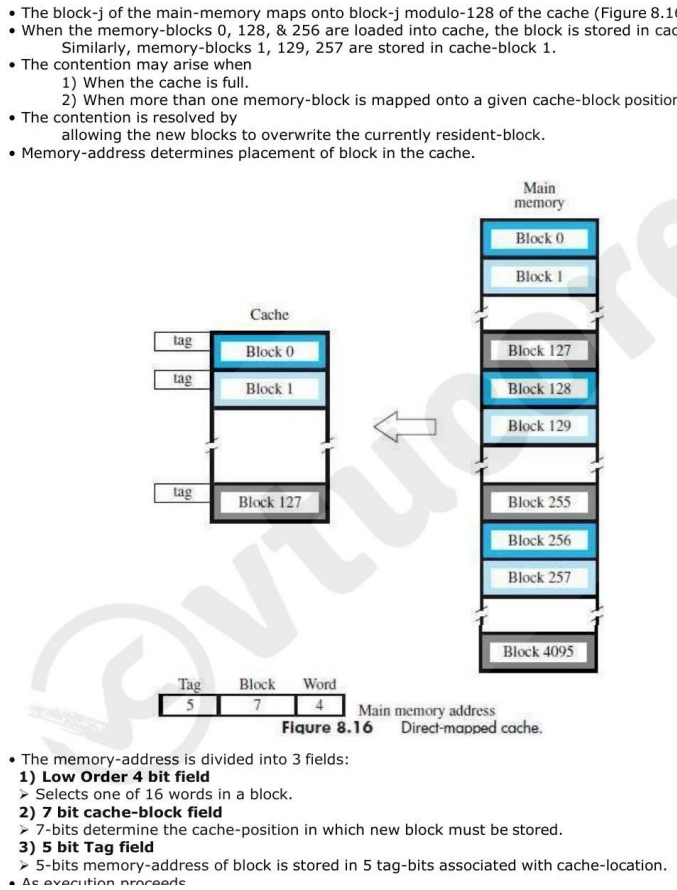

Figure 8.16 — Direct-mapped cache (block j of main memory maps to cache block j mod 128)

Cache lookup:

31. Use the 7-bit block field to find the cache position.
32. Compare the 5-bit tag field of the memory address with the tag stored at that cache position.
33. **Tag match → Cache HIT:** the desired word is in cache. Read directly from cache.
34. **Tag mismatch → Cache MISS:** desired block must be read from main memory and loaded into cache (replacing whatever is currently in that position).

Contention problem: If two frequently-used blocks map to the SAME cache position, they will repeatedly evict each other — called thrashing. Direct mapping is simple but suffers from this inflexibility.

Advantages: Simple hardware, fast lookup (no search needed).

Disadvantage: Fixed mapping causes contention/thrashing if multiple active blocks map to same cache position.

12. Associative Mapping

Rule: A main memory block can be placed into ANY cache block position — completely flexible placement.

Memory address structure (16-word blocks):

- **12-bit Tag:** Identifies the main memory block. All 12 bits needed because any block can go anywhere.
- **4-bit Word:** Selects a word within the block.

ASSOCIATIVE MAPPING

- The memory-block can be placed into any cache-block position. (Figure 8.17).
- 12 tag-bits will identify a memory-block when it is resolved in the cache.
- Tag-bits of an address received from processor are compared to the tag-bits of each block of cache.
- This comparison is done to see if the desired block is present.

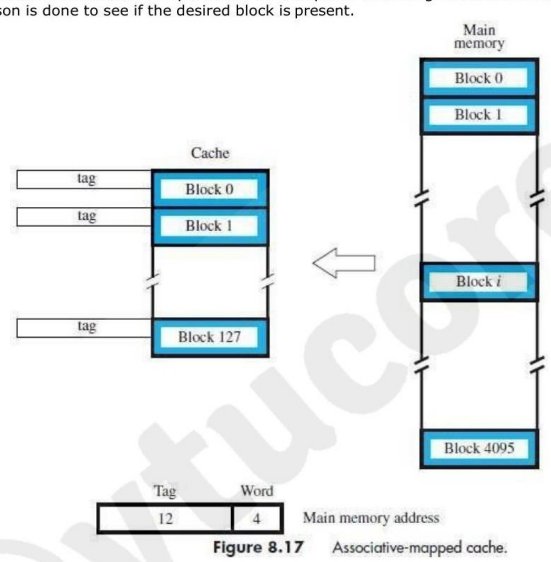

Figure 8.17 Associative-mapped cache.

- It gives complete freedom in choosing the cache-location.
- A new block that has to be brought into the cache has to replace an existing block if the cache is full.
- The memory has to determine whether a given block is in the cache.
- **Advantage:** It is more flexible than direct mapping technique.
- **Disadvantage:** Its cost is high.

Figure 8.17 — Associative-mapped cache (any main memory block → any cache position)

Cache lookup:

35. The 12-bit tag from the processor address is compared SIMULTANEOUSLY to ALL tag entries in the cache.
36. **Tag match found → Cache HIT:** Data is available at that cache position.
37. **No match → Cache MISS:** Block must be fetched from main memory and placed in ANY free cache position. If cache is full, a replacement algorithm selects which block to evict.

Advantage: Most flexible — eliminates the contention problem of direct mapping completely. Any block goes anywhere.

Disadvantage: High cost — requires parallel comparison hardware (associative memory/CAM) to check all tags simultaneously. Cost and power increase with cache size.

13. Set-Associative Mapping

Best of both worlds: Set-associative mapping is a COMBINATION of direct and associative mapping. It eliminates most of the contention of direct mapping while keeping hardware cost much lower than fully associative.

Structure:

- **Cache blocks are grouped into SETS.** Each set contains k blocks (for k-way set-associative cache).
- **A main-memory block maps to a specific SET** (determined by direct mapping) but can occupy ANY BLOCK POSITION within that set (associative within the set).

Example — 2-way set associative (Figure 8.18):

- Cache has 64 sets, each containing 2 blocks → 128 total cache blocks
- Memory blocks 0, 64, 128... 4032 all map to Set 0 (any of the 2 positions in Set 0)
- Memory blocks 1, 65, 129... map to Set 1, and so on

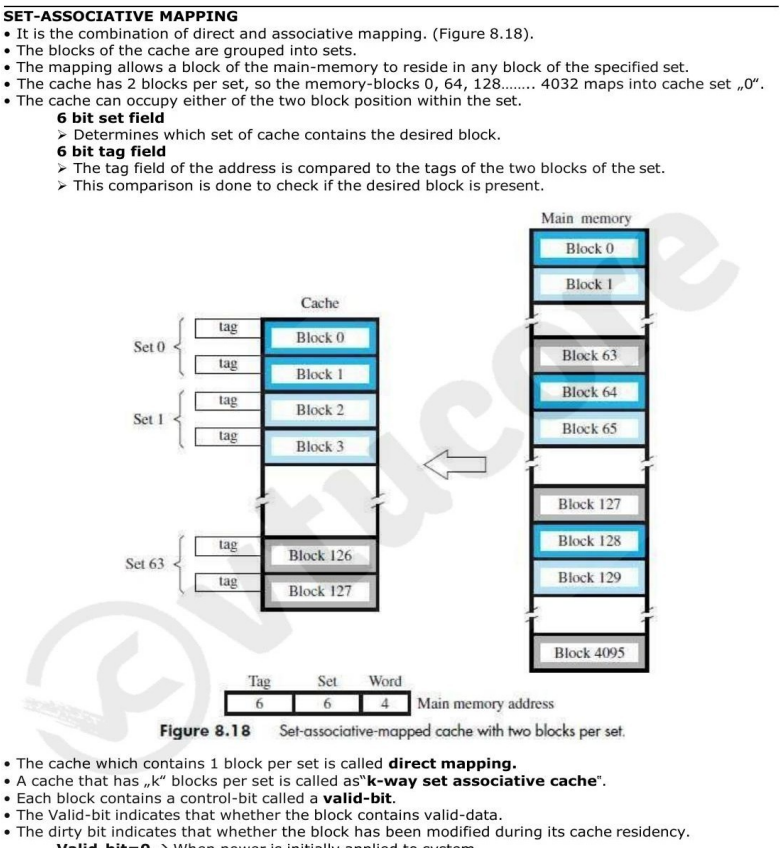

Figure 8.18 — Set-associative-mapped cache (2 blocks per set, 64 sets, Tag=6, Set=6, Word=4)

Memory address structure (Figure 8.18 example):

- **6-bit Tag:** Compared against tags of ALL blocks in the matching set.
- **6-bit Set:** Determines which set of the cache contains the desired block.
- **4-bit Word:** Selects a word within the block.

Special cases:

- **k=1 (1 block per set):** Equivalent to DIRECT MAPPING.
- **k = total number of blocks:** Equivalent to FULLY ASSOCIATIVE MAPPING.

Valid bit: Each block contains a VALID BIT — indicates whether the block contains valid data.

- **Valid = 0:** When power is initially applied — no valid data in cache yet.
- **Valid = 1:** Block loaded from main memory — data is valid.

Dirty bit: Indicates whether the block has been MODIFIED during its stay in cache (used with write-back protocol).

Cache Coherence Problem: If processor and DMA controller use the same copies of data, and DMA updates memory while cache has a stale copy, data inconsistency results. This is the Cache Coherence Problem.

Advantages:

- Solves the contention problem of direct mapping (k choices for block placement)
- Lower hardware cost than fully associative (search only k entries, not all)

14. Replacement Algorithm (LRU)

When is replacement needed?

- **Direct mapping:** NO replacement algorithm needed — position is pre-determined. New block always replaces whatever is in that fixed position.
- **Associative and Set-Associative:** When cache (or a set) is FULL and a new block must be loaded, the cache controller must decide WHICH existing block to evict.

LRU – Least Recently Used Algorithm:

- **Rule:** When a block must be evicted, remove the block that has gone the LONGEST TIME WITHOUT BEING REFERENCED — the one that was least recently used.
- **Implementation:** Cache controller tracks references to all blocks using BLOCK COUNTERS.

LRU with counters (example: 4-way set associative, 2-bit counter per block):

38. **On a HIT:** Set the referenced block's counter to 0. Counters of all blocks that previously had values LOWER than the referenced block are INCREMENTED by 1. Others remain unchanged.
39. **On a MISS (set full):** The block with counter value 3 (LRU block) is REMOVED. The new block is placed in its position with counter = 0. All other block counters are INCREMENTED by 1.

Advantage: LRU performs very well in practice because it exploits temporal locality — recently used blocks are likely to be used again soon.

Other replacement algorithms:

- **FIFO (First In, First Out):** Evict the oldest block (first loaded). Simple but ignores usage patterns.
- **Random Replacement:** Choose a random block to evict. Simple hardware. Performance surprisingly good in practice.
- **Optimal (OPT):** Evict the block that won't be used for the longest time in the future. Theoretically optimal but requires future knowledge — only used as a benchmark comparison.

15. Performance Considerations & Interleaving

Key performance factors:

- How fast machine instructions are brought to the processor (memory bandwidth)
- How fast machine instructions are executed (processor speed)

Interleaving

Problem: Main memory is SLOW compared to the processor and cache. Sequential access to one memory module creates a bottleneck.

Solution — Interleaving: Organize main memory as a collection of PHYSICALLY SEPARATE MODULES, each with its own Address Buffer Register (ABR) and Data Buffer Register (DBR).

How interleaving works:

- **Low-order k bits** of the memory address SELECT A MODULE.
- **High-order m bits** specify the LOCATION WITHIN that module.
- **Consecutive addresses** are in SUCCESSIVE MODULES — not in the same module.
- **Result:** Multiple modules can be accessed simultaneously → higher aggregate memory bandwidth.

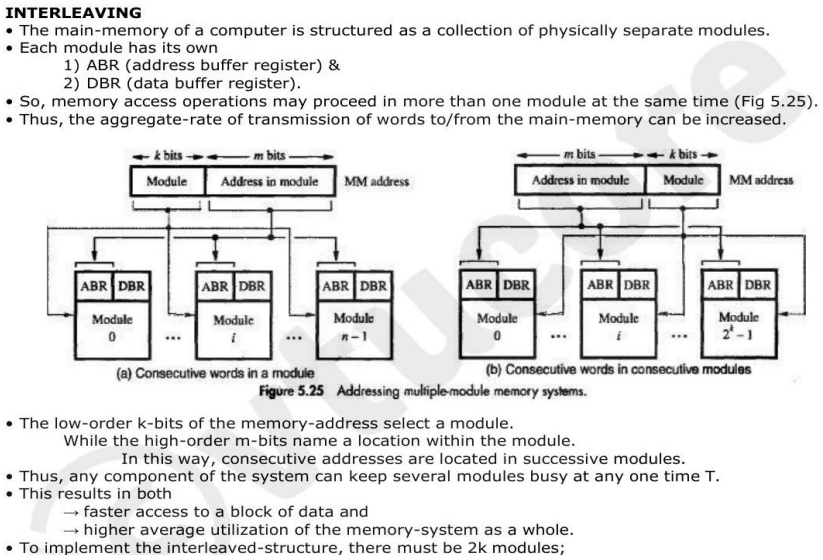

Figure 5.25 — Addressing multiple-module memory systems (a) consecutive words in same module (b) interleaved — consecutive words in consecutive modules

Advantages of interleaving:

- Faster access to a BLOCK of data (modules operate in parallel)
- Higher average utilization of the memory system

Requirement: Must have 2^k modules — otherwise there will be gaps in the address space.

15.1 Hit Rate & Miss Penalty

Hit Rate (h): The fraction of all memory accesses that are found in the cache (served from cache without going to main memory). High-performance systems require h > 0.9.

Miss Rate: = 1 - h. The fraction of accesses that require going to main memory.

Miss Penalty (M): The extra time needed to bring the desired block into cache when a cache miss occurs. This is the total time seen by the processor (including stall time waiting for the block).

Cache Access Time (C): Time to access data from cache when a hit occurs (fast — typically 1-4 clock cycles).

Average Access Time formula:

$$t_{avg} = h \times C + (1-h) \times M$$

Interpretation:

- If h=1 (all hits): t_avg = C (pure cache speed — ideal)
- If h=0 (all misses): t_avg = M (main memory speed — terrible)
- Practical: h=0.95, C=2ns, M=60ns → t_avg = 0.95×2 + 0.05×60 = 1.9 + 3.0 = 4.9ns

VTU Formula: t_avg = hC + (1-h)M. Know this cold — it appears in almost every exam. Also remember: high hit rate (>0.9) is essential for good performance.

16. Virtual Memory

Definition: Virtual memory is a technique that automatically moves program and data BLOCKS into main memory when they are required for execution — programs can be larger than physical main memory.

Key concept — Virtual/Logical vs Physical/Real Address:

- **Virtual Address:** The address generated by the PROCESSOR (what the program sees). Programs work with virtual addresses as if they had a huge memory space.
- **Physical Address:** The ACTUAL address in physical main memory where data is stored.
- **MMU (Memory Management Unit):** Hardware that TRANSLATES virtual addresses to physical addresses on every memory access.

Virtual memory operation:

40. **Processor generates a virtual address.**
41. **MMU looks up the page table** to check if the referenced page is in physical memory.
42. **If page IS in memory:** MMU translates virtual → physical address, memory is accessed, execution continues.
43. **If page is NOT in memory (Page Fault):** The page containing the desired data must be transferred from DISK to memory using DMA. This is a PAGE FAULT.

VIRTUAL MEMORY

- It refers to a technique that automatically move program/data blocks into the main-memory when they are required for execution (Figure 8.24).
- The address generated by the processor is referred to as a **virtual/logical address**.
- The virtual-address is translated into physical-address by **MMU** (Memory Management Unit).
- During every memory-cycle, MMU determines whether the addressed-word is in the memory.
  - If the word is in memory.
    - Then, the word is accessed and execution proceeds.
    - Otherwise, a page containing desired word is transferred from disk to memory.
- Using DMA scheme, transfer of data between disk and memory is performed.

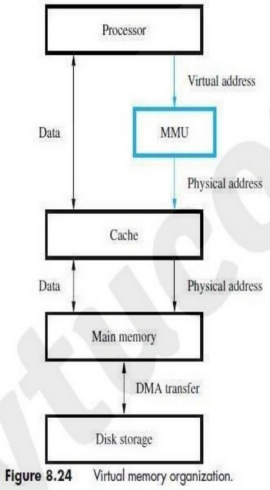

Figure 8.24 Virtual memory organization.

Figure 8.24 — Virtual memory organization (Processor → MMU → Cache → Main Memory → DMA → Disk Storage)

Advantages of virtual memory:

- Programs can be LARGER than physical main memory
- Multiple programs can share physical memory efficiently
- Memory protection — each program has its own virtual address space
- Programmer does not need to manage memory overlays manually

17. Virtual Memory Address Translation

Pages: All programs and data are divided into fixed-length units called PAGES. A page is a contiguous block of memory addresses, typically 2KB to 16KB in size.

Virtual address structure:

- **Virtual Page Number (VPN):** Identifies which page of the program is being accessed.
- **Offset:** Identifies a specific word within the page. The same offset is used in the physical address.

Page Table: A table maintained by the OS (in memory) that contains, for each virtual page:

- **Page Frame Number:** The physical memory location (page frame) where this page is stored.
- **Status/Control bits:** Valid bit (is page in memory?), dirty bit (has page been modified?), protection bits (read-only? read/write?), etc.

Page Frame: An area in physical main memory that holds one page. Physical memory is divided into page-frame-sized regions.

Page-Table Base Register: A register that contains the starting address of the page table for the currently executing process. MMU uses this to find the page table.

Translation process:

44. **Processor generates virtual address (VPN + Offset).**
45. **MMU adds VPN to Page-Table Base Register** → gets address of the page-table entry for this page.
46. **MMU reads the page-table entry** → gets the page frame number AND checks valid bit.
47. **If valid bit = 1 (page in memory):** MMU replaces VPN with page frame number → Physical Address = Page Frame + Offset.
48. **If valid bit = 0 (page not in memory):** PAGE FAULT → OS handles it.

→ current status of the page.
• **Page-frame:** An area in the main-memory that holds one page.
• **Page-table Base Register:** It contains the starting address of the page-table.
• *Virtual Page Number + Page-table Base register* → Gives the starting address of the page if that page currently resides in memory.
• **Control-bits in Page-table:** The Control-bits is used to
1) Specify the status of the page while it is in memory.
2) Indicate the validity of the page.
3) Indicate whether the page has been modified during its stay in the memory.

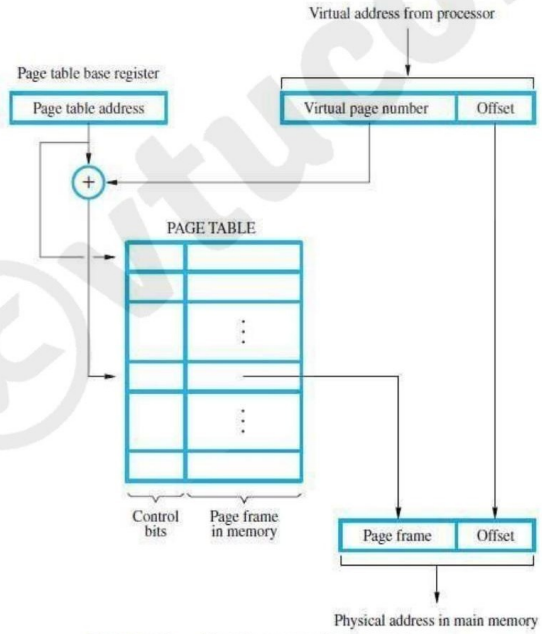

Figure 8.25 Virtual-memory address translation.

3-20

Figure 8.25 — Virtual-memory address translation (Page table lookup: VPN + Offset → Page Frame + Offset)

18. Translation Lookaside Buffer (TLB)

Problem: Page table is stored in main MEMORY. Every memory access requires an EXTRA memory access to read the page table — this would halve performance!

Solution — TLB: A small, fast CACHE for page-table entries, located INSIDE the MMU. The TLB stores the most recently accessed page-table entries.

TLB contents:

- Virtual page number of the entry
- Corresponding page frame number
- Control bits (valid, dirty, protection)

TLB operation (fast path):

49. **MMU checks TLB** for the virtual page number of the current access.
50. **TLB HIT:** Page-table entry found → physical address obtained IMMEDIATELY (no main memory access for translation).
51. **TLB MISS:** Entry not in TLB → read entry from page table in main memory → update TLB → retry.

TRANSLATION LOOKASIDE BUFFER (TLB)

• The Page-table information is used by MMU for every read/write access (Figure 8.26).
• The Page-table is placed in the memory but a copy of small portion of the page-table is located within MMU. This small portion is called **TLB** (Translation LookAside Buffer).
TLB consists of the page-table entries that corresponds to the most recently accessed pages.
TLB also contains the virtual-address of the entry.

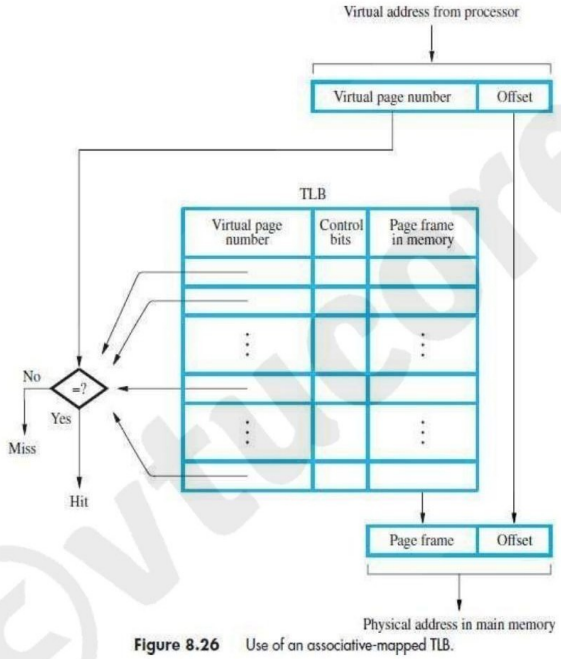

Figure 8.26 Use of an associative-mapped TLB.

• When OS changes contents of page-table, the control-bit will invalidate corresponding entry in TLB.
• Given a virtual-address, the MMU looks in TLB for the referenced-page.
If page-table entry for this page is found in TLB, the physical-address is obtained immediately.
Otherwise, the required entry is obtained from the page-table & TLB is updated.

Page Faults

• Page-fault occurs when a program generates an access request to a page that is not in memory.
• When MMU detects a page-fault, the MMU asks the OS to generate an interrupt.
• The OS
→ suspends the execution of the task that caused the page-fault and
→ begins execution of another task whose pages are in memory.
• When the task resumes the interrupted instruction must continue from the point of interruption.

Figure 8.26 — Use of an associative-mapped TLB (Virtual page number → TLB lookup → Page frame → Physical address)

TLB invalidation: When the OS changes a page-table entry (e.g., after a page fault or context switch), the corresponding TLB entry must be INVALIDATED to prevent using stale translations.

18.1 Page Faults

Definition: A page fault occurs when the processor generates a reference to a page that is NOT currently in physical main memory (valid bit = 0).

Page fault handling sequence:

52. **MMU detects page fault** → signals the OS via an interrupt.
53. **OS SUSPENDS the current task** (saves its complete state: PC, all registers, etc.).
54. **OS starts ANOTHER task** whose pages ARE in memory — so CPU stays productive.
55. **OS initiates DMA transfer** to load the required page from disk into a free page frame in memory.
56. **When DMA completes,** OS is notified (via interrupt) that the page is now in memory.
57. **OS resumes the original task** — the interrupted instruction must be RE-EXECUTED from the beginning (or continued, if processor supports instruction restart).

Page replacement (when memory is full): If no free page frame is available, the OS must EVICT an existing page to make room.

- **LRU algorithm:** Remove the page that has been LEAST RECENTLY REFERENCED.
- **Modified page:** If the evicted page was MODIFIED (dirty bit = 1), it must be written back to disk (write-through) before being removed.

VTU Key: Page fault handling uses context switching — the faulting task is suspended, another runs while disk transfer (via DMA) completes, then the original task resumes. This hides the disk latency.

19. Memory Management Requirements

System Space: The virtual address space where OS routines are assembled. System code is shared among all processes but protected from user access.

User Space: The virtual address space where user application programs reside. Each process has its OWN separate user space with its own page table.

Two processor states:

- **User State:** Processor executes user-program code. Access to privileged instructions is PROHIBITED. Cannot access OS page tables or hardware directly.
- **Supervisor State (Kernel Mode):** Processor executes OS routines. Has FULL ACCESS to all system resources, page tables, and hardware.

Privileged Instructions: Instructions that can ONLY be executed in Supervisor (kernel) mode. A user program attempting to execute a privileged instruction causes a protection violation exception → switches to OS.

Memory Protection via Page Table Control Bits:

- Each page-table entry has PROTECTION BITS that specify what operations are allowed on that page
- Example: OS code pages → read-only for user programs
- Example: User data pages → read/write for the owner process, no access for others

Secondary Storage devices:

- **Magnetic Disk:** Primary secondary storage. Spinning platters with magnetic coating. Slow but large and inexpensive.
- **Optical Disk:** CD-ROM, DVD, Blu-ray. Read-mostly storage.
- **Magnetic Tape:** Sequential access storage for backups and archival.

20. Secondary Storage – Magnetic Hard Disk

Structure: A magnetic disk system consists of one or more DISKS (platters) mounted on a common SPINDLE. Each disk surface is coated with a thin magnetic film.

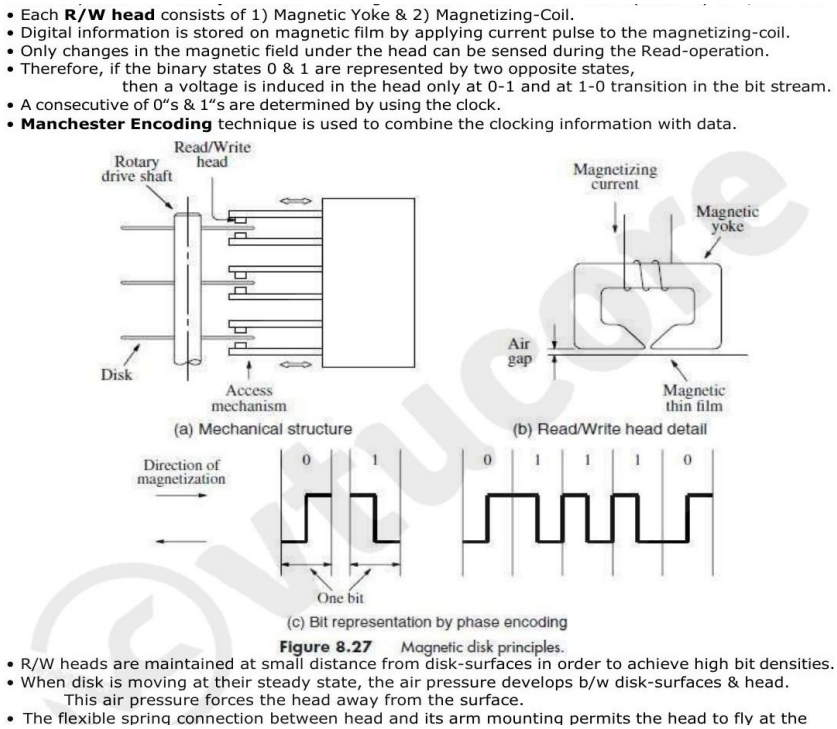

Figure 8.27 — Magnetic disk principles: (a) mechanical structure (b) R/W head detail (c) bit representation by phase encoding (Manchester encoding)

Key components:

- **R/W Head:** Consists of (1) Magnetic Yoke and (2) Magnetizing Coil. Writes by applying current pulses; reads by sensing changes in magnetic field as disk spins under head.
- **Data encoding:** Only CHANGES in magnetic field can be sensed. Manchester Encoding combines clocking information with data — a transition in the middle of each bit cell represents the bit value.
- **Air bearing:** R/W heads "fly" at a very small distance from the disk surface using air pressure developed by the spinning disk. A spring keeps the head at the correct height.

Winchester Technology:

- R/W heads are placed in a SEALED, AIR-FILTERED ENCLOSURE
- Heads can operate closer to the magnetic surface (no dust contamination)
- **Advantages:** Higher capacity for given size, higher data density, heads do not touch surface.

Disk system components:

58. **Disk Platter:** The physical disk(s) with magnetic coating.
59. **Disk Drive:** Spins the disk (motor) and moves R/W heads (actuator arm).
60. **Disk Controller:** Controls the operation of the entire disk system.

21. Disk Organization, Access Time & Controller

Disk geometry terms:

- **Track:** Concentric circular path on a disk surface. Each surface divided into many tracks.
- **Sector:** A pie-slice shaped portion of a track. Each track is divided into sectors. Each sector typically contains 512 bytes + header + ECC bytes.
- **Cylinder:** The set of corresponding tracks on ALL surfaces of a disk stack (same radius). Accessing a cylinder uses the same head position for all surfaces — fast for related data.
- **Logical Cylinder:** The set of corresponding tracks on all surfaces of a stack of disks, forming a virtual cylinder accessed with one head movement.

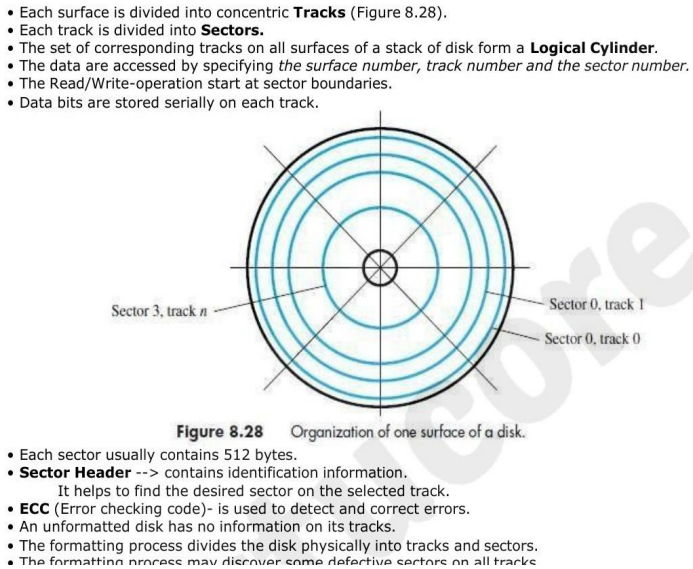

Figure 8.28 — Organization of one surface of a disk (Tracks and Sectors, Cylinders)

Data addressing: Data is accessed by specifying (surface number, track number, sector number). R/W operations start at SECTOR BOUNDARIES. Data bits are stored SERIALLY on each track.

Sector structure:

- **Sector Header:** Contains identification information used to find the sector.
- **Data Area:** 512 bytes of user data (typical).
- **ECC (Error Correcting Code):** Used to detect and correct read errors caused by magnetic defects.

Access Time Components

- **Seek Time:** Time required to MOVE the R/W head to the proper TRACK. Depends on current head position. Typical: 3-10ms.
- **Rotational Latency / Rotational Delay:** After head reaches the correct track, the time for the starting position of the ADDRESSED SECTOR to rotate under the head. Average = half a rotation period. Typical: 3-5ms at 10,000 rpm.

Total Disk Access Time = Seek Time + Rotational Latency

Typical Disk Example:

- Recording surfaces = 20
- Tracks per surface = 15,000
- Sectors per track = 400 (each = 512 bytes)
- Capacity = 20 × 15,000 × 400 × 512 = 60 × 10^9 = 60 GB
- Seek time = 3ms, Rotational speed = 10,000 rpm, Latency = 3ms
- Internal transfer rate = 34 MB/s

Disk Controller Functions

3) Word Count: Number of words in the block to be transferred.

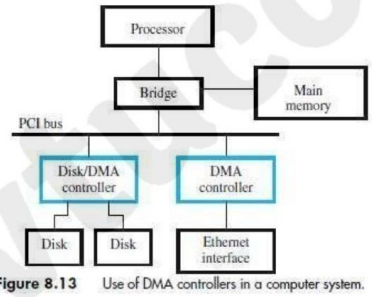

Figure 8.13 Use of DMA controllers in a computer system.

- The disk-address issued by the OS is a logical address.
- The corresponding physical-address on the disk may be different.
- The controller's major functions are:
  - **1) Seek** - Causes disk-drive to move the R/W head from its current position to desired track.
  - **2) Read** - Initiates a Read-operation, starting at address specified in the disk-address register. Data read serially from the disk are assembled into words and placed into the data buffer for transfer to the main-memory.
  - **3) Write** - Transfers data to the disk.
  - **4) Error Checking** - Computes the error correcting code (ECC) value for the data read from a given sector and compares it with the corresponding ECC value read from the disk. In case of a mismatch, it corrects the error if possible; Otherwise, it raises an interrupt to inform the OS that an error has occurred.

Figure 8.13 — Disk controller connecting disk drives to system bus via DMA

The disk controller acts as the interface between the disk drive and the system bus. It uses DMA to transfer data between disk and memory.

When OS issues an R/W request, controller loads:

61. **Memory Address:** Start address in main memory for the data block.
62. **Disk Address:** Logical location of sector on disk.
63. **Word Count:** Number of words in the block to transfer.

Controller major functions:

- **Seek:** Moves R/W head to the required track.
- **Read:** Initiates read from specified disk address; assembles serial bits into words; places in data buffer for DMA transfer to memory.
- **Write:** Transfers data from memory to disk via DMA.
- **Error Checking:** Computes ECC for data read; compares with stored ECC. On mismatch: corrects if possible; else raises interrupt to notify OS of error.

Data Buffer/Cache: The disk drive incorporates a semiconductor CACHE BUFFER. When a read request arrives, controller first checks if data is in the buffer/cache. If yes → serve from cache (fast). If no → retrieve from disk (slow).

22. Solved Problems

Study all 9 problems — they cover all major calculation types in VTU exams.

Problem 1: DRAM Refresh Rate Calculation

Given: C = 30 femtofarads (30×10^{-15} F), leakage current = 0.25 picoamperes (0.25×10^{-12} A), initial voltage = 1.5V, must refresh before voltage drops below 0.9V.

Solution:

Voltage drop allowed = 1.5 - 0.9 = 0.6V

Time = C × ΔV / I_leakage

Time = $(30\times10^{-15} \times 0.6) / (0.25\times10^{-12}) = (18\times10^{-15}) / (0.25\times10^{-12})$

$$\text{Refresh interval} = (30\times10^{-15} \times (1.5-0.9)) / (0.25\times10^{-12}) = 8.33\times10^{-3}\ s \approx 8\ ms$$

Answer: Each row must be refreshed every 8 ms.

Problem 2: SDRAM Transfer Time

Given: SDRAM, burst length = 8, 32 bits transferred in parallel, 400 MHz clock. Find time to transfer (a) 32 bytes, (b) 64 bytes, and latency.

Solution:

Clock period = $1/400\times10^{6}$ = 2.5 ns

Assuming 5 cycles for row activation (latency) + 8 burst cycles = 13 total cycles

(a) 32 bytes = 8 words of 32 bits:

Total time = 13 cycles × 2.5 ns = 32.5 ns ≈ 98 ns (using 133 MHz as $1/(133\times10^{6})$)

Latency = 5 cycles × 2.5 ns = 12.5 ns

(b) 64 bytes = two independent 32-byte transfers:

Time = 2 × 13 cycles = 26 cycles. Latency is the SAME = 5 cycles (per transfer).

Problem 3: Faster Processor vs Memory Speed

Question: "Using a faster processor chip results in a corresponding increase in performance of a computer even if the main-memory speed remains the same." True or false?

Answer: PARTIALLY TRUE but not completely. A faster processor does increase performance, but the increase is NOT directly proportional to the processor speed increase. The reason: the cache MISS PENALTY involves accessing main memory, and if memory speed stays the same, the miss penalty stays the same. So the improvement is limited by the memory bandwidth bottleneck.

Problem 4: Cache Field Bit Calculation

Given: Block-set-associative cache, 64 total blocks, 4-block sets, main memory has 4096 blocks of 128 words each, 32-bit byte-addressable address space.

Solution:

(a) Number of bits in main memory address:

- 4096 blocks: need log2(4096) = 12 bits for block
- 128 words per block: need log2(128) = 7 bits for word offset
- **Total = 12 + 7 = 19 bits for main memory address**

(b) Tag, Set, Word fields:

- 64 total blocks / 4 blocks per set = 16 sets → Set field = $\log_2(16)$ = 4 bits
- Word field = $\log_2(128)$ = 7 bits (within block)
- **Tag field = 19 - 4 - 7 = 8 bits**

Problem 5: Cache Block Size Trade-offs

Question: What are the advantages and disadvantages of larger and smaller cache block sizes?

Larger block size (>128 bytes):

- **Advantage:** Fewer cache misses if most data in the block is actually used (spatial locality works well). Block transfer amortized over more useful data.
- **Disadvantage:** Wasteful if much of the data loaded is NOT used before the block is evicted. Larger blocks take longer to transfer → longer miss penalty. Fewer total cache blocks (cache can hold fewer large blocks).

Smaller block size (<32 bytes):

- **Advantage:** More blocks fit in cache (more blocks total). Transfer is faster. Works well when data access is scattered.
- **Disadvantage:** More cache misses (less spatial locality exploited per miss). More overhead per block (tag, valid bit overhead increases proportionally).

Problem 6: OS Strategy for Minimizing Page Transfers

Given: OS monitors page transfer activity and dynamically adjusts number of pages allocated to programs. Suggest strategy to minimize page transfer rate.

Solution: The OS should INCREASE the number of main-memory pages allocated to programs that have a HIGH PAGE FAULT RATE, using space taken from programs that have a LOW PAGE FAULT RATE. This dynamic working-set adjustment ensures each program has enough pages to avoid thrashing while programs with few page faults give up their underutilized pages.

Problem 7: Page Fault During Instruction Execution

Question: If a page fault interrupts an instruction mid-execution, what state must be saved? Is it simpler to abandon and re-execute the instruction?

Solution:

- **To RESUME the instruction:** Must save the ENTIRE processor state including all registers, plus control information showing HOW FAR instruction execution has progressed (which micro-steps completed). Very complex.
- **To RE-EXECUTE the instruction:** Must UNDO (reverse) any changes made by the partial execution before the page fault. Some instructions may have already written to registers or memory — these must be restored to original values before retry.
- **Practical approach:** Context-switch to another task during the DMA disk transfer, then resume (or re-execute) when the page is available. Most modern processors support instruction restart.

Problem 8: Page Size and Disk Sector Size

Question: Which disk parameter should influence the choice of page size in a virtual memory system?

Solution: The SECTOR SIZE. A sector is the SMALLEST directly addressable block on disk — it is read or written as a unit. Therefore, virtual memory pages should be a small integral multiple of the sector size. This avoids reading/writing partial sectors (which would require read-modify-write operations and waste bandwidth).

Problem 9: Disk Capacity and Transfer Rate

Given: 24 surfaces, 14,000 cylinders, 400 sectors/track, 512 bytes/sector, 7200 rpm.

(a) Maximum capacity:

$$24 \times 14{,}000 \times 400 \times 512 = 68.8 \times 10^9 \text{ bytes} \approx 68.8 \text{ GB}$$

(b) Data transfer rate at 7200 rpm:

$$(400 \times 512 \times 7200) / 60 = 24.58 \times 10^6 \text{ bytes/s} \approx 24.58 \text{ MB/s}$$

(c) 32-bit disk address scheme:

- Sectors = 400 → 9 bits (2^9=512 ≥ 400). Use bits A8-A0
- Cylinders = 14,000 → 14 bits (2^{14}=16384 ≥ 14000). Use bits A22-A9
- Surfaces = 24 → 5 bits (2^5=32 ≥ 24). Use bits A27-A23
- **Bits A31-A28: NOT USED (4 spare bits)**

23. Quick Revision Summary

All key facts, formulas and tables for last-minute exam preparation — vtu-easy.com

Memory Types – Complete Comparison

Feature	SRAM	DRAM (Async)	SDRAM	ROM/Flash
Storage element	Cross-coupled inverters (latch)	Capacitor + transistor	Capacitor + transistor	Transistor + fuse/floating gate
Transistors/ bit	6	1	1	1
Speed	Very fast (ns)	Slower (tens of ns)	Faster than async (clock-sync)	Slow
Refresh needed?	No (static)	Yes (every ~8ms)	Yes	No (non-volatile)
Volatile?	Yes	Yes	Yes	No
Cost	Expensive	Cheap	Cheap	Very cheap
Density	Low (large cells)	High	High	Very high
Use	Cache (L1, L2)	Main memory	Main memory	BIOS, embedded

Cache Mapping – Quick Reference

Feature	Direct Mapping	Associative Mapping	Set-Associative
Block placement	Fixed: block j → cache j mod N	Anywhere in cache	Fixed set, anywhere in set
Tag bits	Few (5 bits in example)	Many (12 bits)	Moderate (6 bits)
Lookup	Single location check	Search all tags in parallel	Search k tags in set
Hardware cost	Low	Very High	Moderate
Contention	Yes (thrashing possible)	No	Minimal
Flexibility	Low	Maximum	Good
Replacement needed?	No	Yes (LRU etc.)	Yes (LRU etc.)

ROM Types – Quick Reference

Type	Full Name	Erase Method	Reprogram?	Notes
PROM	Programmable ROM	Not erasable (irreversible)	No	Fuse burned by user; permanent

EPROM	Erasable PROM	UV light (entire chip)	Yes (remove chip)	Quartz window on chip
EEPROM	Electrically Erasable ROM	Electrical (selective bytes)	Yes (in-circuit)	Different voltages for R/W/Erase
Flash	Flash Memory	Electrical (entire block)	Yes (block by block)	High density; wears out after ~1M writes

Key Formulas

Formula	Description
Memory capacity = 2^k locations	k = number of address bits (MAR width)
t_avg = h×C + (1-h)×M	Average access time; h=hit rate, C=cache time, M=miss penalty
Disk capacity = S×T×Sec×B	Surfaces × Tracks/surface × Sectors/track × Bytes/sector
Transfer rate = (Sec×B×RPM)/60	Disk data transfer rate in bytes per second
Disk access time = Seek + Latency	Latency = rotational delay (average = half rotation time)
DRAM refresh interval = C×ΔV/I_leak	C=capacitance, ΔV=voltage drop allowed, I_leak=leakage current
DDR bandwidth = 2×SDRAM bandwidth	DDR transfers on both clock edges

Virtual Memory Key Terms

Term	Definition
Virtual Address	Address generated by processor — what program sees
Physical Address	Actual location in physical memory — what memory chips see
MMU	Memory Management Unit — hardware that translates virtual to physical addresses
Page	Fixed-size unit of data (2KB-16KB); programs divided into pages
Page Frame	Fixed-size area in physical memory that holds one page
Page Table	OS-maintained table mapping virtual page numbers to page frames
TLB	Translation Lookaside Buffer — fast cache for page table entries inside MMU
Page Fault	Access to a page NOT in physical memory — OS must load it from disk
Page-Table Base Register	Register holding start address of current process page table
LRU Replacement	Evict the page (or cache block) least recently referenced

Frequently Asked VTU Questions

Question (Marks)	Key Points to Cover

Basic concepts of memory (6M)	MAR k-bits→2^k locations, MDR n-bits→n data bits, read/write sequence, access time vs cycle time
Internal organization of memory chip (6M)	Cell array, word-line, sense/write circuit, R/W and CS control, Figure 8.2
Static RAM with read/write operations (8M)	Latch structure, T1/T2 transistors, CMOS cell, read steps, write steps, Figures 8.4 & 8.5
Asynchronous DRAM (8M)	Capacitor storage, refresh need, RAS/CAS multiplexed address, read/write, fast page mode, Figure 8.6 & 5.7
SDRAM with timing diagram (8M)	Clock sync, row/column latches, burst read, Figure 8.8 & 8.9
ROM types comparison (8M)	All 4 types, erase methods, reprogrammability, flash cards/drives
Cache memory and locality (8M)	Temporal/spatial locality, write-through/write-back, read miss, write miss
Direct mapping (6M)	Block j mod 128 rule, tag/block/word fields, hit/miss, Figure 8.16
Associative mapping (4M)	Any block anywhere, 12-bit tag, advantage/disadvantage, Figure 8.17
Set-associative mapping (6M)	Sets + tags, 2-way example, valid bit, dirty bit, Figure 8.18
LRU replacement algorithm (6M)	Counter-based LRU, hit/miss rules, example with 4 blocks
Hit rate and miss penalty (6M)	t_avg = hC + (1-h)M formula and application
Virtual memory (8M)	Virtual vs physical address, MMU, page table, page fault handling
TLB (6M)	TLB structure, hit vs miss, page table interaction, Figure 8.26
Magnetic disk (8M)	Structure, tracks/sectors/cylinders, seek time, latency, Winchester, Figure 8.27 & 8.28
Disk controller (6M)	Memory address, disk address, word count registers, 4 functions (seek/read/write/ECC)
Interleaving (4M)	Physically separate modules, ABR/DBR, low-order bits select module, Figure 5.25
Performance formula problems	t_avg calculation, disk capacity/transfer rate, cache field bits, DRAM refresh